Strategic Decision-Making in Multiagent Domains: A Weighted Constrained Potential Dynamic Game Approach

Maulik Bhatt, *Graduate Student Member, IEEE*, Yixuan Jia, and Negar Mehr, *Member, IEEE*

***Abstract*—In interactive multiagent settings, decision-making and planning are challenging mainly due to the agents' interconnected objectives. Dynamic game theory offers a formal framework for analyzing such intricacies. Yet, solving constrained dynamic games and determining the interaction outcome in the form of generalized Nash equilibria (GNE) pose computational challenges due to the need for solving constrained coupled optimal control problems. In this article, we address this challenge by proposing to leverage the special structure of many real-world multiagent interactions. More specifically, our key idea is to leverage constrained dynamic potential games, which are games for which GNE can be found by solving a single constrained optimal control problem associated with minimizing the potential function. We argue that constrained dynamic potential games can effectively facilitate interactive decision-making in many multiagent interactions. We will identify structures in realistic multiagent interactive scenarios that can be transformed into weighted constrained potential dynamic games (WCPDGs). We will show that the GNE of the resulting WCPDG can be obtained by solving a single constrained optimal control problem. We will demonstrate the effectiveness of the proposed method through various simulation studies and show that we achieve significant improvements in solve time compared to state-of-the-art game solvers. We further provide experimental validation of our proposed method in a navigation setup involving two quadrotors carrying a rigid object while avoiding collisions with two humans.**

***Index Terms*—Dynamic games, multiagent interactions, Nash equilibria, potential games.**

I. Introduction

NUMEROUS robotic applications, such as autonomous driving, crowd–robot navigation, and delivery robots, include scenarios that require multiagent interactions. In these situations, a robot is tasked with engaging with either human individuals or other robots present in the surrounding environment. Making decisions and creating plans in these domains is challenging as agents in multiagent systems may have different goals or objectives. This requires agents to develop strategies that account for how other agents will react to their actions, i.e., each agent needs to reason about the likely reactions of other agents in their decision-making. Furthermore, agents may need to satisfy some task constraints, such as collision avoidance and goal constraints. Such reasoning may be computationally demanding, requiring agents to perform joint prediction and planning.

Dynamic Game theory is a powerful tool for analyzing and understanding multiagent interactions because it provides a formal framework for studying strategic decision-making in situations where an agent's objective may depend not only on its own choices, but also on those of the other agents. In particular, dynamic noncooperative game theory [1] provides a formalism for sequential decision-making involving multiple strategic decision-makers. It has been shown that Nash equilibria of dynamic games can model the interaction outcome among multiple strategic decision makers [2]. Generalized Nash Equilibria (GNE) in dynamic games consists of sets of strategies (formally defined later) for the agents involved in the game such that no agent can benefit more by deviating from it while satisfying the safety constraints. Mathematically, finding the GNE of a constrained dynamic game can be formulated as a set of coupled constrained optimal control problems. However, real-world robot planning problems have nonlinear dynamics, costs, and constraints. Therefore, solving the coupled constrained nonlinear optimal control problems for computing Nash equilibria is a highly challenging task. This computational bottleneck inhibits most of the current game theoretic motion planning methods from being used in real-time.

In this article, we tackle this challenge and provide a framework for tractably finding the equilibria of multiagent interactions that involve strategic decision-makers. Our key idea is that, by utilizing ideas from *potential games* [3], we can significantly simplify the problem of computing the GNE of general-sum dynamic games. Potential games are a class of games for which a potential function exists such that the Nash equilibria of the original game can be computed by optimizing the potential function. Potential games often appear in economics and engineering applications, where multiple agents share a common resource (a raw material, a communication link, a transportation link,

Received 25 June 2024; revised 9 December 2024; accepted 13 February 2025. Date of publication 17 March 2025; date of current version 17 April 2025. This work was supported by the National Science Foundation under Grant ECCS-2438314 CAREER Award, under Grant CNS-2423130 and Grant CCF-2423131. This article was recommended for publication by Associate Editor Stephen L. Smith and Editor David Hsu upon evaluation of the reviewers' comments. *(Corresponding author: Maulik Bhatt.)*.

Maulik Bhatt and Negar Mehr are with the Department of Mechanical Engineering, University of California at Berkeley, Berkeley, CA 94530 USA (e-mail: maulikbhatt@berkeley.edu; negar@berkeley.edu).

Yixuan Jia is with the LIDS, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: yixuany@mit.edu).

This article has supplementary downloadable material available at https://doi.org/10.1109/TRO.2025.3552325, provided by the authors.

Digital Object Identifier 10.1109/TRO.2025.3552325

an electrical transmission line) or limitations. *We argue that in many multiagent interactions, agents either share a space or a task*. Thus, we propose that potential games, particularly in the form of constrained dynamic potential games, can be effectively utilized to facilitate interactive decision-making.

Using constrained potential games, we reduce the coupled constrained optimal control problems of multiagent motion planning into a well-studied single constrained optimal control problem. With this simplification, we can significantly reduce the computational complexity of multiagent motion planning. Using any off-the-shelf optimizer, we can efficiently solve the resulting single optimal control problem, making our algorithm faster and more employable in real-time. However, previous works that utilized potential games for multiagent trajectory planning [4], [5] could only capture symmetric interagent costs or couplings. These approaches limit the types of cost structures that can be captured for multiagent interactions.

In this work, we go beyond symmetric couplings and identify various practical cost structures under which multiagent decision-making and planning turn into weighted constrained potential dynamic games (WCPDGs). We show how the GNE of the underlying dynamic game can be found reliably and tractably for WCPDGs by solving a single constrained optimal control problem. We will show that under a set of mild assumptions, dyadic interactions, i.e., interactions between two agents, are always WCPDGs. We will then consider multiagent settings with more than two agents and identify a set of realistic cost structures under which interactions become WCPDGs. We will discuss how the resulting single constrained optimal control problem can be solved using off-the-shelf solvers such as ALTRO [6], do-mpc [7], and iterative-LQR [8] for efficient multiagent trajectory planning with nonlinear state and action constraints. Through several simulation studies, we will show that our proposed method provides an efficient solution for finding equilibria. To demonstrate the practical relevance of our proposed method, we also showcase a hardware experiment. We will further compare the solve time of our method with the state-of-the-art game solvers and show that our method is significantly faster than the state-of-the-art. In summary, the main contributions of this article are as follows.

1) We find practical cost structures under which the problem of multiagent motion planning problem is a WCPDG.
2) We demonstrate how the GNE of the original game can be computed by optimizing the potential function of the game. This reduces the computational complexity of multiagent motion planning, making the algorithm scalable, faster, and in real time.
3) We show the benefits of our proposed method in a number of simulations, showing efficient and intuitive solutions while outperforming state-of-the-art game solvers in terms of computational speed.
4) We also validate our method in an experiment setting where two drones carried an object while moving around humans.

The rest of this article is organized as follows. Section II provides a literature review on multiagent motion planning. Notations and problem formulation are provided in Section III. In Section IV, we present important results on WCPDG for game theoretic motion planning. Then, we provide realistic cost structures for dyadic and multiagent interactions under which the resulting game will be a WCPDG in Sections V and VI, respectively. Simulations and experiments with the analysis of the resulting performance are incorporated in Sections VII and VIII, respectively. Finally, Section IX concludes this article.

II. Related Work

A. Multiagent Motion Planning

Multiagent motion planning is generally classified into two categories, cooperative and noncooperative, based on how agents interact with each other in the environment. As the name suggests, cooperative motion planning refers to planning in settings where agents are all part of the same team. Various cooperative motion planning algorithms have been used, such as [9], [10], [11], [12], [13]. Multiagent motion planning using buffered voronoi cells was introduced in [14]. There have been various learning-based planners for multiagent motion planning that utilize past trajectory data to improve motion planning in the future. A few representative examples of such methods are [15], [16], [17], [18], [19]. However, learning-based methods generally suffer from nonstationarity in the environment because of simultaneous learning [20], [21]. Furthermore, learning-based methods generally focus on the path-planning aspect and fail to consider the interactive nature of multiagent motion planning problems. Kretzschmar et al. [22] presented an inverse reinforcement learning-based framework for cooperative crowd navigation scenarios. This work explicitly models the interactions between the robot and humans but does not provide a theoretical guarantee of finding equilibrium solutions that abide safety constraints as well. It should be noted that most of the aforementioned methods are cooperative in nature. However, many real-world motion planning scenarios contain agents with individual goals and objectives. Previous works fail to work in such settings. Many game theoretical approaches exist for such noncooperative motion planning problems, which are discussed in the following section.

B. Game-Theoretic Multiagent Interactions

Game theory has proven effective for multiagent systems where agents are individual decision-makers [23]. Due to the interactive nature of multiagent planning, where agents have to interact with each other and take into account the likely reactions of other agents, game theory naturally becomes a robust framework for reasoning about such interdependencies. Seeking Nash equilibria of the underlying game in the interactive multiagent planning domain generates highly intuitive and interactive trajectories.

A Stackelberg equilibrium was initially considered in two-player games for modeling the mutual coupling between two agents [24]. In the context of autonomous driving, a Markovian Stackelberg strategy was developed using hierarchical planning in [25]. However, the proposed dynamic programming approach suffers from the curse of dimensionality. Moreover, Stackelberg

equilibria are not suited to multiagent setups. Stackelberg equilibria commonly involve a leader and a follower; hence, they assume a form of asymmetry among the agents in that one agent reacts to another. A human-like motion planner based on game theoretic decision-making was proposed in [26]. However, to compute Nash equilibria, they generated a random set of sample trajectories and chose the best samples using Pareto optimality, which does not necessarily recover Nash equilibria.

Iterative best-response methods were developed in [27], [28], [29] in the context of autonomous racing. However, such an approach for finding Nash equilibria can be slow because computing the best response for each agent generally involves solving a nonconvex optimization problem. Therefore, Miller and Mitra [30] proposed a Lexicograhpic-based method that helps find approximate Nash equilibria using a linear version of the iterated best response. However, this method does not consider hard constraints on states and actions. Iterative linear quadratic approximations for real-time game theoretic motion planning were introduced in [31]. A belief space-based motion planning method was proposed in [32]. Augmented Lagrangian-based game theoretic solver (ALGAMES) was proposed in [33]. Laine et al. [34] proposed a sequential linear-quadratic technique game-based technique to compute GNE. Seeking equilibria of stochastic variants of general-sum dynamic games was considered in [35] and [36]. A fault-tolerant receding horizon game-theoretic motion planner that leverages interagent communication with intention hypothesis likelihood for multiple highly interactive robots was introduced in [37]. However, these methods are not easily scalable. They struggle to work efficiently in real-time because they attempt to solve complex coupled optimal control problems, which become more difficult as the number of agents increases. In addition, these methods can face convergence issues as the complexity of the game increases with more agents.

C. Potential Games

Potential games have been extensively studied and analyzed, particularly for static games [3], [38]. Due to their simplicity and ease of handling, continuous-time versions of potential games called potential differential games were introduced in [39]. Discrete-time versions of potential differential games are called potential dynamic games. Potential dynamic games and potential differential games have found practical implementations in diverse applications, including resource allocation, cooperative control, power networks, and communication networks, thanks to their simplicity and tractability [40], [41], [42], [43]. Dynamic potential games were recently used in interactive multiagent motion planning [4], [5], [44]. Furthermore, dynamic potential games were utilized in autonomous racing in [45]. However, the type of objectives these methods can capture are very restrictive as they require symmetry in the way agents assign costs to each other.

III. Notation and Preliminaries

We consider the problem of multiagent trajectory planning in a noncooperative setting. We model agents' interactions as a constrained dynamic game. Let $\mathcal{N} = \{1, 2, \ldots, N\}$ be the set of agents' indices, where N is the total number of agents. The state space of each agent $i \in \mathcal{N}$ is denoted by $\mathcal{X}^i \in \mathbb{R}^{n_i}$ where n_i is the dimension of the state space of agent i. Let $\mathcal{X} = \mathcal{X}^1 \times \cdots \times \mathcal{X}^N \in \mathbb{R}^n$ denote the set of states of all agents, where $n = \sum_{i\in\mathcal{N}} n_i$ is the dimension of the state space of the overall system. We assume that the game is played over a finite horizon of time T. Let $x_k = (x_k^{1^\top}, \ldots, x_k^{N^\top})^\top$ denote the state vector of the game at time $k \in \{0, 1, \ldots, T\}$ with $x_k^i \in \mathcal{X}^i$ being the state vector of agent i at time k. Furthermore, let $x^i = \{x_k^i\}_{k\in\{0,1,\ldots,T\}}$ be the collection of states of agent i at all times. Likewise, let $x = \{x_k\}_{k\in\{0,1,\ldots,T\}}$ be the collection of the overall system state vector at all times.

For each agent, $i \in \mathcal{N}$, let $\mathcal{U}^i \subseteq \mathbb{R}^{m_i}$ denote the action space of agent i, where m_i is the dimension of its action space. The action space of the overall game is denoted by $\mathcal{U} = \mathcal{U}^1 \times \cdots \times \mathcal{U}^N \subseteq \mathbb{R}^m$ where $m = \sum_{i\in\mathcal{N}} m_i$. The vector of agents' actions at time k is denoted by $u_k := (u_k^{1^\top}, \ldots, u_k^{N^\top})^\top \in \mathcal{U}$ where $u_k^i \in \mathcal{U}_k^i$ is the vector of agent i's actions at time k. Similar to before, we define $u^i = \{u_k^i\}_{k\in\{0,1,\ldots,T-1\}}$ and $u = \{u_k\}_{k\in\{0,1,\ldots,T-1\}}$ to be the collection of action vectors for agent i and the overall system respectively over the entire horizon of time. Furthermore, for every agent $i \in \mathcal{N}$, we define $u_k^{-i} := (u_k^{1^\top}, \ldots, u_k^{i-1^\top}, u_k^{i+1^\top}, \ldots, u_k^{N^\top})^\top \in \mathcal{U}^{-i}$ as the vector of all agents' actions at time k except agent i, where $\mathcal{U}^{-i} := \Pi_{j\neq i}\mathcal{U}^j$. We also have $u^{-i} = \{u_k^{-i}\}_{k\in\{0,1,\ldots,T-1\}}$. With a slight abuse of notation, we can write $u_k = (u_k^{i^\top}, u_k^{-i^\top})^\top$.

We assume that each agent's dynamics depend only on its own states and actions via

$$x_{k+1}^i = f^i(x_k^i, u_k^i). \tag{1}$$

Note that this is a valid assumption as most real-world robotic interactions contain robots that have separable dynamics, and the couplings among the agents normally arise from the couplings inherent in agents' objectives. The overall system dynamics for the game are defined by $f : \mathcal{X} \times \mathcal{U} \to \mathcal{X}$

$$x_{k+1} = f(x_k, u_k) \tag{2}$$

where $f = (f^{1^\top}, \ldots, f^{N^\top})^\top$. It should be noted that we have assumed time-invariant dynamics for notational simplicity, but this formulation can easily be extended to time-varying dynamics as well.

In order to account for constraints, we assume that there is an inequality constraint function $g : \mathcal{X} \times \mathcal{U} \to \mathbb{R}^c$ where c is the number of constraints. The inequality constraints are defined as

$$g(x_k, u_k) \leq 0 \tag{3}$$

where the inequality is interpreted elementwise. For the terminal time-step, the inequality constrained is only a function of the terminal state, $g(x_T) \leq 0$. It is to be noted that any equality constraint can also be included as a combination of two inequality constraints. Also, we assume that constraints that capture coupling between agents are shared among the agents. Furthermore, it should be noted that we have assumed time-invariant constraints for notional simplicity, but this formulation can easily be extended to time-varying constraints

as well. Let $\mathcal{C}_k$ at each time step k be the set of all the feasible states x_k and actions u_k. We can define $\mathcal{C}_0 = \mathcal{X} \times \mathcal{U} \cap \{(x_0, u_0) : g(x_0, u_0) \leq 0\}$ and $\mathcal{C}_k = \{\{\mathcal{X} \cap \{x_k \mid x_k = f(x_{k-1}, u_{k-1})\}\} \times \mathcal{U}\} \cap \{(x_k, u_k) : g(x_k, u_k) \leq 0\}$ for $k \in \{1, \ldots, T-1\}$. Furthermore, at the terminal time step T, no action is being taken, and hence, the constraint function will only be a function of states, and the corresponding constraint set will be $\mathcal{C}_T = \mathcal{X} \cap \{x_T : g(x_T) \leq 0\}$.

We assume that each agent i seeks to minimize a cost function $J^i : \mathcal{X} \times \mathcal{U}^i \to \mathbb{R}$ that can depend on the full state of the system and the agent's actions itself

$$J^i(x, u) = L^i_T(x_T) + \sum_{k=0}^{T-1} L^i_k(x_k, u^i_k) \tag{4}$$

where $L^i_T(\cdot)$ and $L^i_k(\cdot)$ are the terminal and running costs of agent i, respectively. The decomposition of cost into running and terminal components offers a clear interpretation of trade-offs in decision-making. The running cost reflects immediate consequences, aiding consideration of short-term impacts. The terminal cost captures long-term objectives and provides an intuitive measure of the final outcome. Also, it should be noted that for each agent i, we consider costs that depend only on the actions of agent i and the overall system state because we consider agents with individual objectives like reaching a goal, using minimal control efforts while avoiding collision with others. Therefore, the costs depend on the overall system state that includes other agents' states, but each agent does not care about the control effort of others, so the costs depend only on its own controls.

The goal of each agent is to choose its control actions such that its cost function is minimized while satisfying constraints. The policies through which the control actions are chosen at each time instant are called agents' strategies. We denote the strategy of each agent at time k by $\gamma^i_k \in \Gamma^i$ where Γ^i is the strategy space of agent i. The strategy of each agent depends on the information available to the agent at that time. Let $\eta^i_k \subseteq \{x, u\}$ be the information gained and recalled by agent i at time k. Depending on the information structure, each agent chooses actions according to their policy $\gamma^i_k(\eta^i_k) = u^i_k$. We consider open-loop strategies that are only a function of the system's initial state and time. Therefore, we have the following relation, $\gamma^i(x_0, k) := u^i_k$.

We would like to point out that in this article, we focus on finding open-loop equilibria of the dynamic game, which implies that at equilibrium, the entire trajectory of each agent is the best response to the trajectories of all the other agents for a given initial state of the system, i.e., the control action of every agent is only a function of time step k and the initial state. We will repeatedly solve for open-loop Nash equilibria in a receding-horizon fashion to adapt to new information obtained over time and mimic a feedback policy. We acknowledge that for general dynamic games, due to the difference between the information structure of open-loop and feedback Nash equilibria, the two concepts may result in different behaviors [46]. However, for many trajectory planning purposes, a receding-horizon implementation of open-loop equilibria results in reasonable approximations [33]. Furthermore, we would like to point out that this formulation requires the assumption that all agents have knowledge of the dynamics and objectives of other agents. While knowledge of the dynamics of others is a reasonable assumption, several recent works such as [36] have shown that such estimates of objectives can be obtained through inverse game theoretic learning methods.

We denote the combined strategy of all agents at each time k by $\gamma_k := (\gamma^{1^\top}_k, \ldots, \gamma^{N^\top}_k)^\top \in \Gamma$ where $\Gamma := \Gamma^1 \times \cdots \times \Gamma^N$ is the strategy space of the entire game at time k. Similarly, we define $\gamma^i := \{\gamma^i_k\}_{k \in \{0,1,\ldots,T-1\}}$ and $\gamma := \{\gamma_k\}_{k \in \{0,1,\ldots,T-1\}}$. Moreover, same as before, for each agent $i \in \mathcal{N}$, we denote $\gamma_k = (\gamma^{i^\top}_k, \gamma^{-i^\top}_k)^\top$ where $\gamma^{-i}_k = (\gamma^{1^\top}_k, \ldots, \gamma^{i-1^\top}_k, \gamma^{i+1^\top}_k, \ldots, \gamma^{N^\top}_k)^\top$ is the tuple containing strategies of all agents except agent i. We further define $\gamma^{-i} := \{\gamma^{-i}_k\}_{k \in \{0,1,\ldots,T-1\}}$ to be the strategy of all agents except agent i over the entire horizon of time. Note that a given strategy γ identifies a unique set of actions for all time instants, $\{u_k\}_{k \in \{0,1,\ldots,T-1\}}$, and; therefore, there is an equivalence between the two ($\gamma \equiv \{u_k\}_{k \in \{0,1,\ldots,T-1\}}$). Hence, for simplicity, we use strategies and actions interchangeably from now on. Furthermore, in the case of open-loop strategies, actions at all time steps can be determined by the system's initial state, and a given strategy will, in turn, determine the states of the system at each time step. Therefore, the state x_k can be written as a function of strategies, initial states, and the time step, i.e., $x_k \equiv x_k(\gamma, x_0)$. We do not denote this dependence when it is obvious from the context, but we denote it explicitly when required. Furthermore, it should be noted that not all strategies will result in trajectories that satisfy all the constraints. Therefore, we formally define feasible strategies as follows.

Definition 1: Given an initial state $x_0 \in \mathcal{C}_0$, a strategy $\gamma = (\gamma^1, \ldots, \gamma^N) \in \Gamma$ is a feasible strategy if the system trajectory generated by using control inputs obtain through γ satisfies

$$(x_k, \gamma(x_0, k)) \in \mathcal{C}_k, k \in \{0, \ldots, T-1\}, x_T \in \mathcal{C}_T. \tag{5}$$

Furthermore, we call such a system trajectory $\{x, u\}$ to be a feasible system trajectory.

Intuitively, a strategy is called a feasible strategy if the trajectory generated by applying the strategy to the system satisfies the constraints.

Given an initial state of the system x_0, we represent the resulting dynamic game in a compact form as $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i \in \mathcal{N}}, \{J_i\}_{i \in \mathcal{N}}, \{\mathcal{C}_k\}_{k \in \{0,\ldots,T\}}, f, x_0)$. We seek the open-loop GNE (OLGNE) of the dynamic game underlying multiagent interactions. The OLGNE of the game is defined as:

Definition 2: An OLGNE of a game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i \in \mathcal{N}}, \{J_i\}_{i \in \mathcal{N}}, \{\mathcal{C}_k\}_{k \in \{0,\ldots,T\}}, f, x_0)$ is a feasible strategy $\gamma^* = (\gamma^{1^*}, \ldots, \gamma^{N^*}) \in \Gamma$ for which the following holds for each agent $i \in \mathcal{N}$ and for all feasible strategies $(\gamma^i, \gamma^{-i^*}) \in \Gamma$

$$J^i(x_0, \gamma^*) \leq J^i(x_0, \gamma^i, \gamma^{-i^*}). \tag{6}$$

Intuitively, no agent can decrease their cost function at equilibrium by unilaterally changing their strategies to any other feasible strategy. In GNE, the strategies of the agents are further coupled due to the joint constraints of the agents. It is important

to note that solving (6) involves solving a set of N coupled constrained optimal control problems, which are typically difficult to solve efficiently for robotic systems with large numbers of agents with general nonlinear dynamics, cost and constraint functions.

IV. Constrained Potential Dynamic Games

In this section, we focus on a particular class of games called constrained potential dynamic games in which the difference in costs of each agent can be captured by the difference in a function called the potential function. A potential function assigns a value to each possible outcome of the game, such that any change in the strategy of one agent results in a change in the potential function. This property makes it relatively easy to analyze the agents' behavior and predict the game's outcome. In particular, in the case of constrained potential dynamic games, finding the OLGNE of the problem at hand can be linked to solving a single constrained optimal control problem, the solutions of which correspond to the OLGNE in the original dynamic game. Our key insight is that the solution to a multiagent trajectory optimization problem can be seen as a constrained dynamic potential game, and its OLGNE can be found by solving a single constrained optimal control problem. Based on the type of potential function and their relation to agents' cost functions, there are various types of potential games in the literature, such as exact, weighted, and ordinal potential games [3].

Static versions of exact potential games were defined in [3]. The following definition formally defines the dynamic version of exact constrained potential dynamic games.

Definition 3: A given game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ is an exact constrained potential dynamic game if there exists a potential functional of the form $P(x,u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k,u_k)$ such that for every agent $i \in \mathcal{N}$ and for every pair of feasible open-loop strategies $\gamma^i, \nu^i \in \Gamma^i$, we have the following relation when the strategy of all the other agents are fixed to be feasible strategies $\gamma^{-i} \in \Gamma^{-i}$

$$J^i(x,u) - J^i(x',u') = P(x,u) - P(x',u') \tag{7}$$

where (x,u) is the feasible system trajectory generated by strategies $\{\gamma^i, \gamma^{-i}\}$, and (x',u') denotes the feasible system trajectory generated by $\{\nu^i, \gamma^{-i}\}$.

Intuitively, condition (7) implies that a game is a potential game if the change in the costs of each agent when they change their policy can be captured by a joint potential function when the strategies of all the other agents are fixed.

In our previous works [4], [5], we discussed how exact potential games can be employed for efficient and fast multiagent motion planning. However, due to the strict conditions in Definition 3, the type of cost functions considered for agents are very limited in the case of exact potential dynamic games. For example, it can only capture costs where the cost terms that couple agents' decisions among one another are symmetric and the same among the agents. We will relax the strict assumptions in our previous work by considering a more general class of potential dynamic games.

We will consider WCPDGs, which can capture various realistic cost functions for multiagent planning. Let $w = (w_k^i)_{i\in\mathcal{N},\, 0\le k\le T}$ be a set of positive numbers which we will call *weights*. Taking motivation from [3], we define WCPDGs in the present context as follows:

Definition 4: A given game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ is a WCPDG with positive weights $w = (w_k^i)_{i\in\mathcal{N},\, 0\le k\le T}$ if there exists a potential functional of the form $P(x,u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k,u_k)$ such that for every agent $i \in \mathcal{N}$ and for every pair of feasible open loop strategies $\gamma^i, \nu^i \in \Gamma^i$, we have the following when the feasible strategy of all the other agents are fixed to be $\gamma^{-i} \in \Gamma^{-i}$

$$\begin{aligned} J^i(x,u) - J^i(x',u') &= w_T^i(L_T(x_T) - L_T(x'_T)) \\ &\quad + \sum_{k=0}^{T-1} w_k^i(L_k(x_k,u_k) - L_k(x'_k,u'_k)) \end{aligned} \tag{8}$$

where (x,u) is the feasible system trajectory generated by strategies $\{\gamma^i, \gamma^{-i}\}$, and (x',u') denotes the feasible system trajectory generated by $\{\nu^i, \gamma^{-i}\}$.

In a nutshell, the condition in (8) suggests that the game is a WCPDG if the change in the combined running and terminal costs for each agent can be reflected in the changes to the potential function P with its running and terminal costs multiplied by fixed positive weights. Essentially, there exists a potential function P that can gauge the rate of change in an agent's cost when the strategies of the other agents remain unchanged.

We consider the following assumptions for the rest of the article:

Assumption 1: Agents' running costs L_k^i and terminal costs L_T^i are continuously differentiable on $\mathcal{X}\times\mathcal{U}$ and $\mathcal{X}$, respectively.

Assumption 2: The systems dynamics f and constraints g are continuously differentiable in $\mathcal{X}\times\mathcal{U}$ and satisfy some regularity conditions (such as the linear independence of gradients or the Mangasarian-Fromovitz constraint qualification).

In the following, we discuss essential circumstances under which a game will be a WCPDG. This provides us with mathematical tools to examine if a game is a potential game for a given set of agents' individual cost functions. The following lemma provides the necessary and sufficient conditions under which a dynamic game is a WCPDG.

Lemma 1: A noncooperative dynamic game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ is a WCPDG with positive weights $w = (w_k^i)_{i\in\mathcal{N},\, 0\le k\le T}$ if and only if there exists a potential function $P(x,u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k,u_k)$ such that for every agent $i \in \mathcal{N}$, every time step $0 \le k \le T-1$, and every feasible trajectory $\{x,u\}$ we have

$$\begin{aligned} \frac{\partial L_k^i(x_k,u_k)}{\partial x_k^i} &= w_k^i\left(\frac{\partial L_k(x_k,u_k)}{\partial x_k^i}\right) \\ \frac{\partial L_k^i(x_k,u_k)}{\partial u_k^i} &= w_k^i\left(\frac{\partial L_k(x_k,u_k)}{\partial u_k^i}\right) \end{aligned} \tag{9}$$

and for the final time step, we have

$$\frac{\partial L_T^i(x_T)}{\partial x_T^i} = w_T^i \left(\frac{\partial L_T(x_T)}{\partial x_T^i} \right), \ \forall i \in \mathcal{N}. \tag{10}$$

Proof: From (9), for each agent $i \in \mathcal{N}$ and for all $k = 0, 1, \ldots, T-1$, we have that

$$\begin{aligned} \frac{\partial}{\partial x_k^i} \left(L_k^i(x_k, u_k) - w_k^i(L_k(x_k, u_k)) \right) &= 0 \\ \frac{\partial}{\partial u_k^i} \left(L_k^i(x_k, u_k) - w_k^i(L_k(x_k, u_k)) \right) &= 0 \end{aligned} \tag{11}$$

and from (10), for the final time step, we have

$$\frac{\partial}{\partial x_T^i} \left(L_T^i(x_T) - w_T^i(L_T(x_T)) \right) = 0.$$

This means that the difference between each agent's costs and the game potential function should not depend on x_k^i and u_k^i. Therefore, we can rewrite the above condition as

$$L_k^i(x_k, u_k^i, u_k^{-i}) = w_k^i L_k(x_k, u_k^i, u_k^{-i}) + \Theta_k^i(x_k^{-i}, u_k^{-i}) \tag{12}$$

for every $0 \leq k \leq T-1$, and

$$L_T^i(x_T) = w_T^i L_T(x_T) + \Theta_T^i(x_T^{-i}) \tag{13}$$

for some functions $\{\Theta_k^i\}_{k \in \{0,1,\ldots,T\}}$. Let u^i and u'^i be the collection of action vectors for agent i corresponding to the different feasible strategies γ^i and ν^i. Furthermore, let the state trajectories of agent i generated by those strategies be x^i and x'^i. It should be noted that since all agents' dynamics are separate and we are keeping the strategies of all the other agents except agent i to be the same, x^{-i} and u^{-i} will remain the same. Therefore, (12) and (13) holds true for all $u_k^i \in \mathcal{U}^i$ and we have

$$L_k^i(x_k', u'^i_k, u_k^{-i}) = w_k^i L_k(x_k', u'^i_k, u_k^{-i}) + \Theta_k^i(x_k^{-i}, u_k^{-i}) \tag{14}$$

for every $0 \leq k \leq T-1$, and

$$L_T^i(x_T') = w_T^i(L_T(x_T')) + \Theta_T^i(x'^{-i}_T). \tag{15}$$

By subtracting (14) from (12) and summing over all k and adding the difference of (15) from (13), we obtain (8) which completes if direction of the proof.

For the only if part, taking the partial derivative of (8) with respect to x_k^i and u_k^i for $0 \leq k \leq T-1$, we obtain (9) for every agent $i \in \mathcal{N}$. Similarly, taking the partial derivative of (8) with respect to x_T^i, we obtain (10) for all $i \in \mathcal{N}$. This completes the proof of the other direction. □

Lemma 1 essentially suggests that a game is a WCPDG if and only if, for every agent, the partial derivative of its cost function with respect to their states and actions are a scaled version of the partial derivatives of the potential function with respect to the agent's states and actions. We can rephrase these conditions through the following lemma, which suggests that a game is a WCPDG if every agent's cost function can be decomposed into two terms where one term represents the potential function, and the other term is independent of the states and actions of the agent itself.

Lemma 2: A game $\mathcal{G}$ is a WCPDG with positive weights $w = (w_k^i)_{i \in \mathcal{N},\, 0 \leq k \leq T}$ if and only if the running costs and terminal costs of every agent $i \in \mathcal{N}$ can be expressed as the sum of a term that is common to all agents plus another term that depends neither on its own action nor on its own state components, i.e., for every agent $i \in \mathcal{N}$ and for all feasible trajectories $\{x, u\}$, we have the following for some function $\Theta_k^i(\cdot)$ for $0 \leq k \leq T-1$ and $\Theta_T^i(\cdot)$ for $k = T$:

$$\begin{aligned} L_k^i(x_k, u_k^i, u_k^{-i}) &= w_k^i(L_k(x_k, u_k^i, u_k^{-i})) + \Theta_k^i(x_k^{-i}, u_k^{-i}) \\ L_T^i(x_T) &= w_T^i(L_T(x_T)) + \Theta_T^i(x_T^{-i}). \end{aligned} \tag{16}$$

Proof: By taking the partial derivatives of (16) with respect to x_k^i and u_k^i for $0 \leq k \leq T-1$ and x_T^i, we obtain (9) and (10). Therefore, we can apply Lemma 1 to prove that $\mathcal{G}$ is a WCPDG. This completes the if direction of the proof. Now, assume that $\mathcal{G}$ is a WCPDG. Therefore, conditions of Lemma 1 hold true. As proven in Lemma 1, the conditions of Lemma 1 leads to (14) and (15) that are same as (16). This completes the only if direction of the proof, completing the proof. □

An important property of potential dynamic games is that a set of GNE can be found by solving a single constrained multivariable optimal control problem. We state this important property in the following result.

Theorem 1: Suppose that $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i \in \mathcal{N}}, \{J_i\}_{i \in \mathcal{N}}, \{\mathcal{C}_k\}_{k \in \{0,\ldots,T\}}, f, x_0)$ is a WCPDG, and in addition, Assumptions 1 and 2 hold. Then, a solution to the multivariable optimal control problem

$$\begin{aligned} \underset{\gamma \in \Gamma}{\text{minimize}} \quad & L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k, u_k) \\ \text{subject to} \quad & x_{k+1} = f_k(x_k, u_k) \\ & g_k(x_k, u_k) \leq 0 \end{aligned} \tag{17}$$

is an OLGNE for $\mathcal{G}$.

Proof: The proof is provided in Appendix A. □

In contrast to multiple constrained coupled optimal control problems, Theorem 1 essentially provides us with a single constrained optimal control problem that we can solve for finding the OLGNE of a WCPDG. The obtained single-constrained optimal control problem can be solved efficiently using any off-the-shelf optimizer. This significantly reduces the computational complexity of finding OLGNE. It should be noted that one does not need a centralized coordinator to solve (17) and provide OLGNE trajectories to all the agents. Each agent keeps a copy of (17) and solves it independently to obtain the OLGNE and extract their individual controls out of the solution. We acknowledge that in practice, there could be scenarios where multiple OLGNEs could exist, and some kind of strategy alignment mechanism might be required for such scenarios. This has been studied in recent work [47]. We plan to extend our work to account for multiple OLGNEs however, it is beyond the scope of the current work.

In the following sections, we characterize conditions under which a constrained dynamic game is a WCPDG and show that a large number of realistic multiagent interactions can be modeled as potential dynamic games.

V. Dyadic Interactions

In this section, we consider dyadic interactions, which involve interactions between two agents in a noncooperative setting. We will show that under certain conditions, the underlying dynamic game of two-agent interactions is always a WCPDG. Therefore, optimizing the potential function can efficiently solve the resulting game. We formalize these conditions in this section.

In this case, we denote the agents by indices 1 and 2. For the two agent case, the state and control input of the system at time k will be $x_k = ({x_k^1}^\top, {x_k^2}^\top)^\top$ and $u_k = ({u_k^1}^\top, {u_k^2}^\top)^\top$. Let the stage-wise cost of both agents be given by

$$L_k^1(x_k, u_k) = L_k^{11}(x_k^1, u_k^1) + c_k^1 L_k^{12}(x_k^1, x_k^2) \tag{18a}$$

$$L_k^2(x_k, u_k) = L_k^{22}(x_k^2, u_k^2) + c_k^2 L_k^{21}(x_k^2, x_k^1) \tag{18b}$$

for $0 \leq k \leq T-1$, and

$$L_T^1(x_T) = L_T^{11}(x_T^1) + c_T^1 L_T^{12}(x_T^1, x_T^2) \tag{19a}$$

$$L_T^2(x_T) = L_T^{22}(x_T^2) + c_T^2 L_T^{21}(x_T^2, x_T^1) \tag{19b}$$

for the final time step $k = T$. Note that $c_k^1 > 0, c_k^2 > 0, k \in \{0, 1, \ldots, T\}$ are hyper parameters in the agents' cost functions. Intuitively, for each agent, the stage-wise cost is a sum of agent-specific cost ($L_k^{ii}(\cdot)$) and the interagent cost ($L_k^{ij}(\cdot)$). We assume that interagent costs do not depend on agents' actions. Agent-specific cost terms can be considered as terms that capture costs such as goal reaching, accomplishing certain individual tasks, satisfying individual preferences, avoiding static obstacles, controlling effort penalties, etc. Interagent costs are cost terms that capture the coupling between the two agents, such as preferences for staying out of close proximity of the other agent. It should be noted that even though we have collision avoidance constraints encoded in $g(\cdot, \cdot)$, we observe that different agents have different preferences for how close they get to the other agents in the environment. We consider the following assumption about the interagent costs that capture the coupling among the agents.

Assumption 3: For every time step $1 \leq k \leq T$, we assume that $L_k^{12}(x_k^1, x_k^2) = L_k^{21}(x_k^2, x_k^1)$ for every tuple of states (x_k^1, x_k^2).

Assumption 3 states that the functional form of the terms of the interagent costs is the same for both agents. This is a valid assumption as we have coefficients c_k^1 and c_k^2 to capture the asymmetries between the agents on how they want to consider the interagent costs. For example, suppose the interagent cost term captures the proximity preference cost between the two agents. In that case, generally, such costs are captured by distance functions, which are the same for all the agents. However, if one agent cares more about avoiding close proximity than the other agent, that agent will have a higher weight on the proximity preference cost than the other.

We present the following lemma, which states that such dyadic interactions are always WCPDGs.

Lemma 3: A dyadic interaction with stage wise costs (18) and (19) is always a WCPDG with weights $w_k^1 = \frac{1}{c_k^2}$ and $w_k^2 = \frac{1}{c_k^1}$ if Assumption 3 holds. Furthermore, the corresponding potential function is $P(x, u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k, u_k)$ where

$$L_k(x_k, u_k) = c_k^2 L_k^{11}(x_k^1, u_k^1) + c_k^1 L_k^{22}(x_k^2, u_k^2) + c_k^1 c_k^2 L_k^{12}(x_k^1, x_k^2) \tag{20}$$

and

$$L_T(x_T) = c_T^2 L_T^{11}(x_T^1) + c_T^1 L_T^{22}(x_T^2) + c_T^1 c_T^2 L_T^{12}(x_T^1, x_T^2). \tag{21}$$

Proof: From (18a) we have that

$$\begin{aligned} L_k^1(x_k, u_k) &= L_k^{11}(x_k^1, u_k^1) + c_k^1 L_k^{12}(x_k^1, x_k^2) \\ &= \frac{1}{c_k^2}\left(c_k^2 L_k^{11}(x_k^1, u_k^1) + c_k^1 c_k^2 L_k^{12}(x_k^1, x_k^2)\right) \\ &= \frac{1}{c_k^2} L_k(x_k, u_k) - \frac{c_k^1}{c_k^2} L_k^{22}(x_k^2, u_k^2) \\ &= w_k^1 L_k(x_k, u_k) + \Theta_k^1(x_k^2, u_k^2) \end{aligned} \tag{22}$$

where $w_k^1 = \frac{1}{c_k^2}$ and $\Theta_k^1(x_k^2, u_k^2) = -\frac{c_k^1}{c_k^2} L_k^{22}(x_k^2, u_k^2)$. Similarly, from Assumption 3 we can write,

$$L^2(x_k, u_k) = w_k^2 L_k(x_k, u_k) + \Theta_k^2(x_k^1, u_k^1) \tag{23}$$

where, $w_k^2 = \frac{1}{c_k^2}$, $\Theta_k^2(x_k^1, u_k^1) = -\frac{c_k^2}{c_k^1} L_k^{11}(x_k^1, u_k^1)$. Likewise, for the final time step, T, we have

$$L_T^1(x_T) = w_T^1 L_T(x_T) + \Theta_T^1(x_T^2) \tag{24}$$

$$L_T^2(x_T) = w_T^2 L_T(x_T) + \Theta_T^2(x_T^1) \tag{25}$$

where $\Theta_T^1(x_T^2) = -\frac{c_T^1}{c_T^2} L_T^{22}(x_T^2)$, and $\Theta_T^2(x_T^1) = -\frac{c_T^2}{c_T^1} L_T^{11}(x_T^1)$. Therefore, applying Lemma 2, we have that such a dyadic interaction will be a WCPDG. □

As mentioned previously, this result extends the symmetry assumptions required for the interagent costs from our previous work to arbitrary values of cost coefficients in Lemma 3 for dyadic interactions. This captures many real-world robotics interactions where there is an asymmetry in the way agents assign costs to each other. The following section discusses a more general version of the result in Lemma 3 for multiagent interactions.

VI. Multiagent Interactions

In this section, we generalize our results from the previous section and go beyond two agents. We characterize conditions under which the interaction of more than two agents is a WCPDG.

Consider the interactions of N agents in an environment. For every agent $i \in \mathcal{N}$, let the stage-wise and terminal costs be of the form

$$L_k^i(x_k, u_k) = L_k^{ii}(x_k^i, u_k^i) + \sum_{\substack{j=1 \\ j \neq i}}^{N} c_k^{ij} L_k^{ij}(x_k^i, x_k^j) \tag{26a}$$

and

$$L_T^i(x_T) = L_T^{ii}(x_T^i) + \sum_{\substack{j=1 \\ j \neq i}}^{N} c_k^{ij} L_T^{ij}(x_T^i, x_T^j). \tag{26b}$$

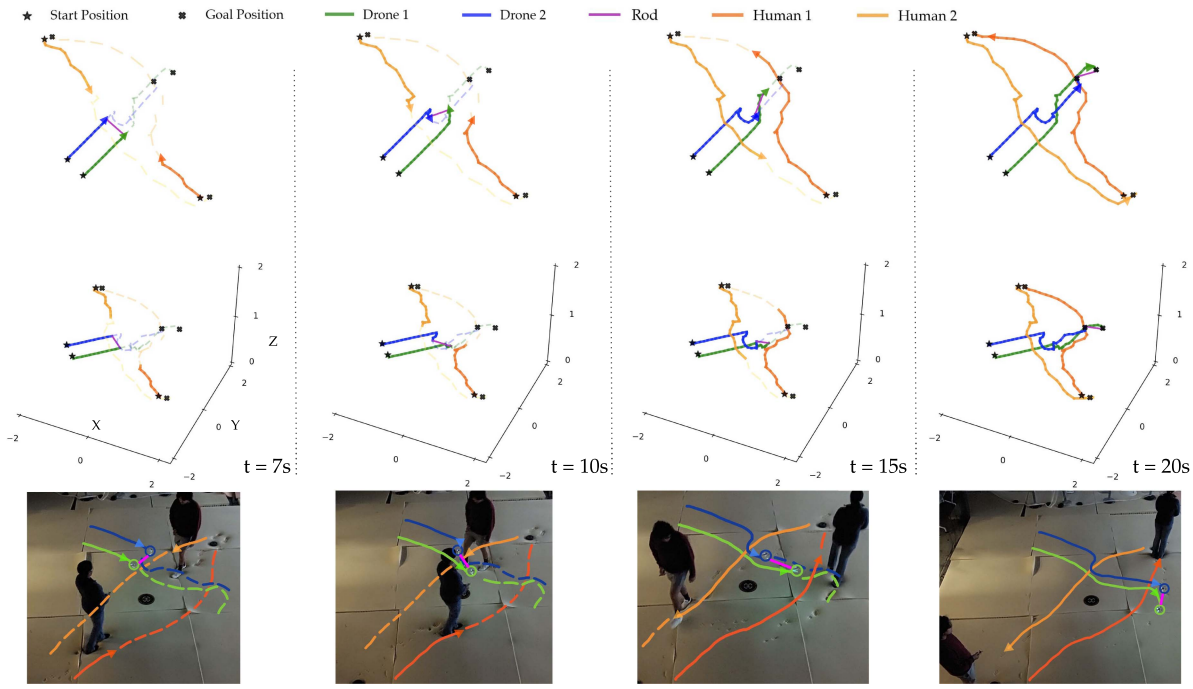


Fig. 1. Visualizations and the corresponding snapshots of the hardware experiment. The two quadrotors and two humans need to reach their designated goal positions while avoiding collision with others. The two quadrotors are carrying a rod. In this case, each quadrotor needs to account for the motions of humans as well as the other quadrotor, as they share the constraint imposed by the rod. As shown in the figure, with our algorithm, the two quadrotors are able to avoid nearby humans by changing their orientations elegantly while carrying the rod. (Refer to Appendix B for the approval details for conducting experiments with human subjects.) We have also included a supplementary video of the experiment which will be available at https://ieeexplore.ieee.org.

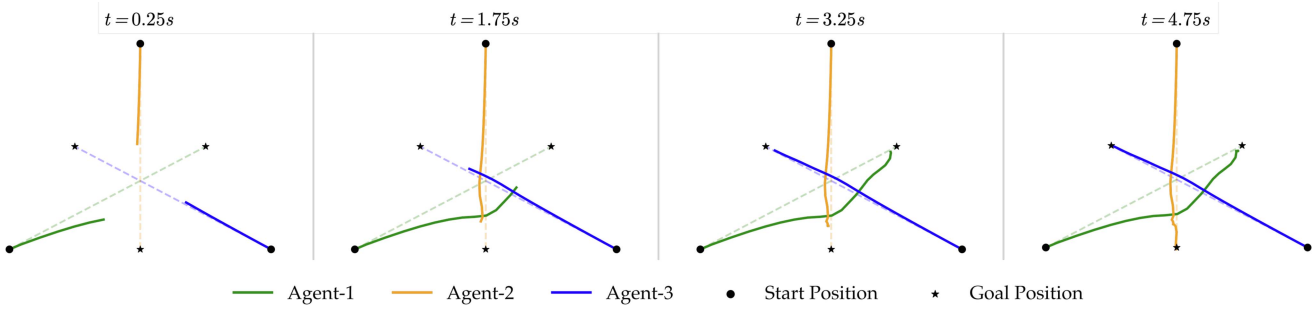


Fig. 2. Snapshots of the trajectories found by our algorithm when three unicycle agents interact with each other and exchange their positions diagonally over a time interval of 5 s. The cost functions are highly asymmetric, and Agent 1 incurs more costs for getting close to other agents. Agents move from their start positions to their respective goal positions. Dashed lines represent the nominal path for each agent. Because of the asymmetric nature of the interaction, Agent 1 yields to other agents and deviates more from its nominal path to move towards its goal.

We also assume that $c_k^{ij} > 0$ for all $i, j \in \mathcal{N}$ and for all $k \in \{0, 1, \ldots, T\}$. Intuitively, the stage-wise cost for each agent is the sum of a term that depends on the agent's own state and actions $L_k^{ii}(x_k^i, u_k^i)$ and the sum of terms that capture the pairwise interaction of the agent i with other agents j, $L_k^{ij}(x_k^i, x_k^j)$. As mentioned earlier, the first term can be thought of as a term that captures the individual preferences of the agent while the coupling between the agents is captured through the other terms. In the following lemmas, we show that under certain conditions, such multiagent interactions become WCPDGs. Same as before, we make an additional assumption that interagent cost terms have the same functional form among the agents.

Assumption 4: Assume that for any two agents $i, j \in \mathcal{N}, i \neq j$, and every time step $1 \leq k \leq T-1$, we have that $L_k^{ij}(x_k^i, x_k^j) = L_k^{ji}(x_k^j, x_k^i)$ for any $x_k^i \in \mathcal{X}^i$ and $x_k^j \in \mathcal{X}^j$., Likewise, for the final time step, we have $L_T^{ij}(x_T^i, x_T^j) = L_T^{ji}(x_T^j, x_T^i)$ for any set of final states $x_T^i \in \mathcal{X}^i$ and $x_T^j \in \mathcal{X}^j$.

Similar to the previous section, the above assumption requires that the functional form of the couplings is the same among the agents, and we capture the asymmetries between the agents through the interagent cost coefficients (c_k^{ij}).

In order to better explain our ideas, we consider a running example of three agents interacting with each other in an environment. Fig. 3 provides a schematic figure to represent this

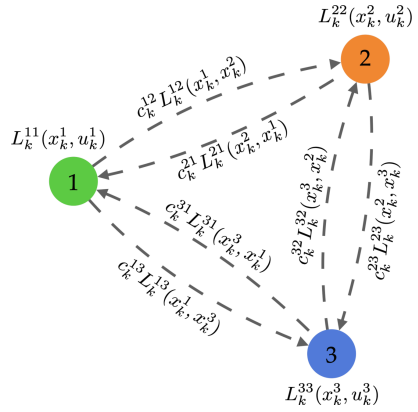


Fig. 3. Schematic of the cost function dependencies for a running example with three agents interacting with one another.

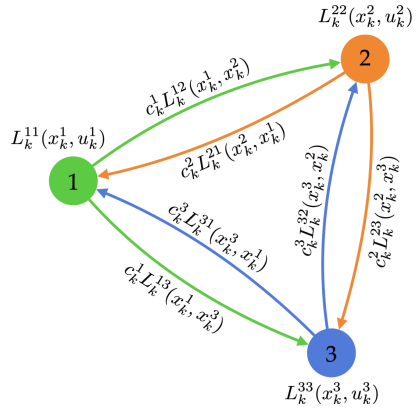


Fig. 4. Schematic of an interaction where each agent in the environment treats all the other agents in a similar fashion. In such scenarios, the underlying dynamic game becomes a weighted potential dynamic game. Note that in this figure, the edges with the same color indicate the same value of the cost coefficient.

interaction. Each agent has two types of costs: agent-specific and interagent costs. The interagent costs are shown on directed arrows, which show the cost a particular agent incurs from the other agent. For example, the arrow from agent 1 to agent 2 captures the cost $c_k^{12} L_k^{12}(x_k^1, x_k^2)$ that is the cost incurred to agent 1 from agent 2. Consider a scenario of multiagent interactions where each agent incurred a cost for getting in close proximity with all the other agents in the environment with the same coefficient, i.e., $c^{ij} = c^i$. This means that each agent has the same interagent cost coefficient for all the other agents in the environment, i.e., *each agent treats all the other agents similarly*. The schematic of such an interaction is shown in Fig. 4. In the following lemma, we show that such an interaction becomes an instance of a w-potential game and, therefore, we can associate a potential function with it, minimizing which results in fast and efficient trajectory planning.

Lemma 4: Under Assumption 4, a multiagent interaction game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ with stage-wise and terminal costs (26a) and (26b) is a w-potential game with weights

$$w_k^i = \frac{1}{\Pi_{\substack{j=1\\ j\neq i}}^{N} c_k^j} \tag{27}$$

if for every agent i in $\mathcal{N}$, we have $c_k^{ij} = c_k^i$ for all $j \in \mathcal{N} \setminus \{i\}$. The corresponding potential function of the game is $P(x,u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k, u_k)$ where

$$L_k(x_k, u_k) = \sum_{i=1}^{N} (\Pi_{\substack{j=1\\ j\neq i}}^{N} c_k^j) L_k^{ii}(x_k^i, u_k^i) + (\Pi_{l=1}^{N} c_k^l) \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} L_k^{ij}(x_k^i, x_k^j) \tag{28}$$

and

$$L_T(x_T) = \sum_{i=1}^{N} (\Pi_{\substack{j=1\\ j\neq i}}^{N} c_k^j) L_T^{ii}(x_T^i) + (\Pi_{l=1}^{N} c_k^l) \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} L_T^{ij}(x_T^i, x_T^j). \tag{29}$$

Proof: From (26a) and Assumption 4, we have that

$$L_k^i(x_k, u_k) = L_k^{ii}(x_k^i, u_k^i) + c_k^i \sum_{\substack{j=1\\ j\neq i}}^{N} L_k^{ij}(x_k^i, x_k^j)$$
$$= \frac{1}{\Pi_{\substack{j=1\\ j\neq i}}^{N} c_k^j} \left[(\Pi_{\substack{j=1\\ j\neq i}}^{N} c_k^j) L_k^{ii}(x_k^i, u_k^i) + (\Pi_{l=1}^{N} c_k^l) \sum_{\substack{j=1\\ j\neq i}}^{N} L_k^{ij}(x_k^i, x_k^j) \right]. \tag{30}$$

Therefore,

$$L_k^i(x_k, u_k) = w^i L_k(x_k, u_k) - w^i \sum_{\substack{j=1\\ j\neq i}}^{N} (\Pi_{\substack{l=1\\ l\neq j}}^{N} c_k^l) L_k^{jj}(x_k^j, u_k^j) - w^i (\Pi_{l=1}^{N} c_k^l) \sum_{\substack{j=1\\ j\neq i}}^{N-1} \sum_{\substack{l=j+1\\ l\neq i}}^{N} L_k^{jl}(x_k^j, x_k^l)$$
$$= w^i L_k(x_k, u_k) + \Theta_k^i(x_k^{-i}, u_k^{-i}) \tag{31}$$

where $\Theta_k^i(x_k^{-i}, u_k^{-i}) = -w^i \sum_{j=1, j\neq i}^{N} (\Pi_{\substack{l=1\\ l\neq j}}^{N} c_k^l) L_k^{jj}(x_k^j, u_k^j) - w^i (\Pi_{l=1}^{N} c_k^l) \sum_{j=1, j\neq i}^{N-1} \sum_{l=j+1, l\neq i}^{N} L_T^{jl}(x_T^j, x_T^l)$. Similarly, we can write

$$L_T^i(x_T) = w^i (L_T(x_T)) + \Theta_T^i(x_T^{-i}) \tag{32}$$

where $\Theta_T^i(x_T^{-i}) = -w^i \sum_{j=1, j\neq i}^{N} (\Pi_{\substack{l=1\\ l\neq j}}^{N} c_k^l) L_T^{jj}(x_T^j) - w^i (\Pi_{l=1}^{N} c_k^l) \sum_{j=1, j\neq i}^{N-1} \sum_{l=j+1, l\neq i}^{N} L_T^{jl}(x_T^j, x_T^l)$. Therefore, by applying Lemma 2, we have that such a multiagent interaction is a WCPDG. □

In the following, we consider another practical case that is of relevance to many real-world applications. Consider a scenario of multiagent interactions where each agent is penalized similarly by all the other agents in the environment. This means that each agent is treated with the same interagent cost coefficient

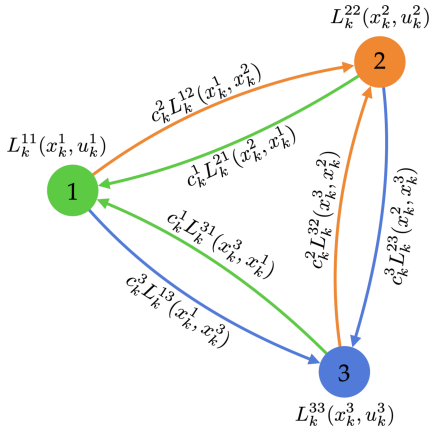


Fig. 5. Schematic of interactions where each agent in the environment is treated by all the other agents in a similar fashion. In such scenarios, the underlying dynamic game becomes a weighted potential dynamic game. It should be noted that edges with the same color indicate the same value of the cost coefficient.

by all the other agents in the environment, i.e., $c^{ij} = c^j$ for all $i \in \mathcal{N} \setminus \{j\}$. The schematic of this type of interaction is shown in Fig. 5.

In the following lemma, we show that such an interaction also becomes an instance of a w-potential game; therefore, we can associate a potential function with it.

Lemma 5: Under Assumption 4, a multiagent interaction game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ with stage-wise and terminal costs according to (26a) and (26b) is a w-potential game with weights $w^i = \frac{1}{c_k^i}$ if $c_k^{ij} = c_k^j$ for every $i \in \mathcal{N} \setminus \{j\}$. The corresponding potential function is $P(x,u) = L_T(x_T) + \sum_{k=0}^{T-1} L_k(x_k, u_k)$ where

$$L_k(x_k,u_k) = \sum_{i=1}^{N} c_k^i L_k^{ii}(x_k^i, u_k^i) + \sum_{i=1}^{N-1}\sum_{j=i+1}^{N} c_k^i c_k^j L_k^{ij}(x_k^i, x_k^j) \tag{33}$$

and

$$L_T(x_T) = \sum_{i=1}^{N} c_k^i L_T^{ii}(x_T^i) + \sum_{i=1}^{N-1}\sum_{j=i+1}^{N} c_k^i c_k^j L_T^{ij}(x_T^i, x_T^j). \tag{34}$$

Proof: From (26a) and Assumption 4, we have that

$$\begin{aligned} L_k^i(x_k,u_k) &= L_k^{ii}(x_k^i,u_k^i) + \sum_{\substack{j=1\\ j\neq i}}^{N} c_k^j L_k^{ij}(x_k^i, x_k^j) \\ &= \frac{1}{c_k^i}\left\{ c_k^i L_k^{ii}(x_k^i,u_k^i) + \sum_{\substack{j=1\\ j\neq i}}^{N} c_k^i c_k^j L_k^{ij}(x_k^i, x_k^j) \right\}. \end{aligned} \tag{35}$$

Therefore,

$$\begin{aligned} L_k^i(x_k,u_k) &= w^i L_k(x_k,u_k) - \frac{1}{c_k^i}\left\{ \sum_{\substack{j=1\\ j\neq i}}^{N} L_k^{jj}(x_k,u_k) \right. \\ &\quad \left. + \sum_{\substack{j=1\\ j\neq i}}^{N-1}\sum_{\substack{l=j+1\\ l\neq i}}^{N} L_k^{jl}(x_k^j, x_k^l) \right\} \\ &= w^i L_k(x_k,u_k) + \Theta_k^i(x_k^{-i}, u_k^{-i}) \end{aligned} \tag{36}$$

where $\Theta_k^i(x_k^{-i}, u_k^{-i}) = -\frac{1}{c_k^i}\{\sum_{\substack{j=1\\ j\neq i}}^{N} L_k^{jj}(x_k,u_k) + \sum_{\substack{j=1\\ j\neq i}}^{N-1}\sum_{\substack{l=j+1\\ l\neq i}}^{N} L_k^{jl}(x_k^j, x_k^l)\}$. Similarly, we can write

$$L_T^i(x_T) = w^i(L_T(x_T)) + \Theta_T^i(x_T^{-i}), \tag{37}$$

where $\Theta_T^i(x_T^{-i}) = \frac{1}{c_k^i}\{\sum_{\substack{j=1\\ j\neq i}}^{N} L_T^{jj}(x_T) + \sum_{\substack{j=1\\ j\neq i}}^{N-1}\sum_{\substack{l=j+1\\ l\neq i}}^{N} L_T^{jl}(x_T^j, x_T^l)\}$. Therefore, applying Lemma-2, we have that such a multiagent interaction will be a WCPDG. □

Remark 1: Note that in this formulation, all agents are not necessarily required to interact with all other agents in the environment. For example, if agent i and agent j do not interact with each other, then the corresponding interagent cost term $L_k^{ij}(x_k^i, x_k^j)$ will be zero, and; therefore, that term won't appear in the potential function.

Lemmas 3–5 provide realistic motion planning scenarios under which a game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\dots,T\}}, f, x_0)$ is a WCPDG. Therefore, we can use the result from Theorem 1 and associate a single constrained optimal control problem as described in (17) for finding equilibria of the game. If there are no constraints in the game, then solving (17) is an easy task as it can be solved using any standard optimization techniques of single agent optimal control. In particular, in unconstrained settings, we employ the iterative linear quadratic regulator (iLQR) [8] to solve the single agent optimal control problem. As the name suggests, the iLQR is an algorithm that iteratively refines both the control inputs and the value function estimates to improve the control policy. Furthermore, in constrained settings, to solve (17), we use the Augmented Lagrangian Trajectory Optimization (ALTRO) algorithm [6]. ALTRO combines iLQR [8] with an augmented Lagrangian method to handle general state and input constraints and an active-set projection method for final "solution polishing" to achieve fast and robust convergence.

VII. Simulations Studies

In this section, we provide the simulation results for our algorithm. First, we provide an analysis of two-player LQ games to show that our algorithm indeed recovers the open-loop Nash equilibrium of the original game. Then, we provide analysis for general nonlinear games to showcase the capabilities of our algorithm in realistic scenarios. Furthermore, we show through Monte Carlo analysis that our algorithm is faster than state-of-the-art methods.

A. Dyadic Linear Quadratic Interactions

The main result of our work is to prove that we can compute the open-loop Nash equilibrium of a game by minimizing a single potential function if a game is WCPDG. In order to numerically verify this claim, we consider a two-agent Linear Quadratic Game (LQ Game). We choose this because, for linear quadratic games, exact closed-form solutions for computing open-loop Nash equilibria are available. LQ games are a special version of affine-quadratic games ([1, Definition 6.1]), where the dynamics are entirely linear with no affine term

Definition 5: A game is a linear quadratic (LQ Game) if the system dynamics are defined as

$$x_{k+1} = f_k(x_k, u_k) = A_k x_k + B_k u_k \tag{38}$$

where A_k is the state matrix and B_k is the control matrix; and the stage-wise cost functions for every agent $i \in \mathcal{N}$ are

$$L_k^i(x_k, u_k) = \begin{cases} \frac{1}{2}\left(\sum_{j\in\mathcal{N}} {u^j}_k^\top R_k^{ij} u_k^j\right), & k = 0 \\ \frac{1}{2}\left(x_k^\top Q_k^i x_k + \sum_{j\in\mathcal{N}} {u^j}_k^\top R_k^{ij} u_k^j\right), & k \neq 0, T \end{cases} \tag{39}$$

and

$$L_T^i(x_T) = \frac{1}{2} x_T^\top Q_T^i x_T \tag{40}$$

where Q^i's are symmetric positive semi-definite matrices, and R^{ii}'s are positive definite matrices.

Now, we provide an example of a potential linear quadratic game and show how potential minimization recovers open-loop Nash equilibrium. Consider a two player LQ Game - $\mathcal{G}^{LQ}$ where $x^1 := (x^{11}, x^{12})^\top$ and $x^2 := (x^{21}, x^{22})^\top$ are state vector for both the agents with x^{ij} being jth element of ith agent. We let the initial conditions of the game be $x_0^1 = (3, 2)^\top$ and $x_0^2 = (4, 5)^\top$. Let the dynamics and cost matrices be

$$A_k = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & -1 \end{bmatrix}, \quad B_k = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$Q_k^1 = \begin{bmatrix} 1 & -1 & 2 & 0 \\ -1 & 5 & -1 & 1 \\ 2 & -1 & 6 & -2 \\ 0 & 1 & -2 & 4 \end{bmatrix}, \quad Q_k^2 = \begin{bmatrix} 1 & -1 & 2 & 0 \\ -1 & 4 & -1 & 1 \\ 2 & -1 & 6 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix}$$

$$R_k^{11} = \begin{bmatrix} 3 \end{bmatrix}, R_k^{12} = 0, \qquad R_k^{21} = 0, R_k^{12} = \begin{bmatrix} 2 \end{bmatrix}. \tag{41}$$

It is easy to verify using Lemma 2 that $\mathcal{G}^{LQ}$ is a WCPDG with the weights $w_1 = w_2 = 1$ and stage-wise potential functions

$$L_k(x_k, u_k) = \begin{cases} \frac{1}{2}\left(u_k^\top R_k u_k\right), & k = 0 \\ \frac{1}{2}\left(x_k^\top Q_k x_k + u_k^\top R_k u_k\right), & k \neq 0, T \end{cases} \tag{42}$$

and

$$L_T(x_T) = \frac{1}{2} x_T^\top Q_T x_T \tag{43}$$

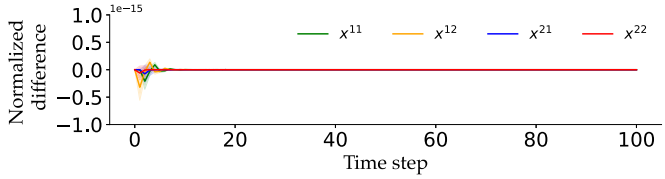


Fig. 6. Comparison of trajectories for the LQ-game solved using potential minimization and exact open-loop Nash solution shows that the trajectories generated are close up to the system precision level.

where, R_k and Q_k are

$$Q_k^1 = \begin{bmatrix} 1 & -1 & 2 & 0 \\ -1 & 5 & -1 & 1 \\ 2 & -1 & 6 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix}, \quad R_k = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}. \tag{44}$$

The open-loop Nash equilibria of the LQ game can be found using [1, Th. 6.2]. By formulating the game as a w-potential game, we need to just solve the resulting single-agent optimal control problem, which can be solved using Ricatti equations of standard discrete-time LQR. We consider 200 random initial conditions around x_0^1 and x_0^2. We plot the mean and standard deviation of the trajectories that result from potential function minimization and the exact open-loop trajectories of the system in Fig. 6. As we can observe from Fig. 6, the trajectories obtained from Potential minimization and open-loop Nash equilibrium of the original game are close to the system precision level. This demonstrates that, indeed, we can recover the open-loop Nash equilibrium of the original game by potential function minimization for the case of w-potential game.

B. Multiagent Interactive Planning

In order to evaluate the performance of our method on systems with general nonlinear dynamics and costs, we consider an interaction among three agents in a planner navigation setting. Each agent wants to reach its goal position while avoiding collisions with other agents. We model each agent via discrete-time unicycle dynamics with a time step size of Δt

$$p_{k+1}^i = p_k^i + \Delta t \cdot v_k^i \cos\theta_k^i, \quad q_{k+1}^i = q_k^i + \Delta t \cdot v_k^i \sin\theta_k^i$$
$$\theta_{k+1}^i = \theta_k^i + \Delta t \cdot \omega_k^i, \quad v_{k+1}^i = v_k^i + \Delta t \cdot \alpha_k^i \tag{45}$$

where p_k^i and q_k^i are x and y coordinates of the positions in the 2-D plane, θ_k^i is the heading angle from positive x-axis, v_k^i is the forward velocity, ω_k^i is the angular velocity, and α_k^i is the acceleration of agent i at time step k. For each agent i, we let the state vector x_k^i be $[p_k^i, q_k^i, \theta_k^i, v_k^i]$, and the action vector u_k^i be $[\omega_k^i, \alpha_k^i]$. For each agent i, we consider costs of the form (26a) and (26b) with conditions from Lemma 4. Here, $L^{ii}(\cdot)$ consists of the distance to goal cost and control costs while $L^{ij}(\cdot)$ consists of collision avoidance costs. We choose interagent collision avoidance costs to be of the form

$$L_k^{ij}(x_k^i, x_k^j) = \mathbf{1}\{d(x_k^i, x_k^j) < d_m\} \cdot \left(d(x_k^i, x_k^j) - d_m\right)^2 \tag{46}$$

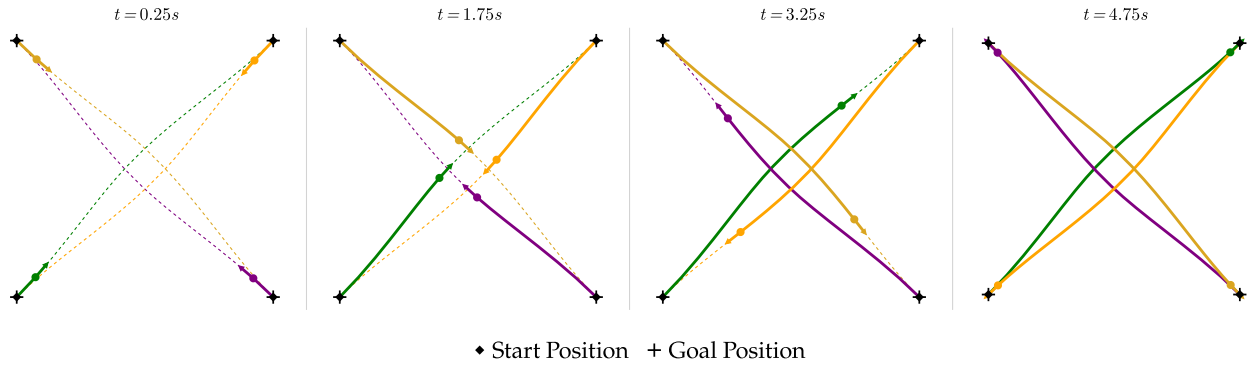


Fig. 7. Snapshots of the trajectories found by our algorithm when four unicycle agents interact with each other and exchange their positions diagonally over a time interval of 5s. Agents move from their start positions to their respective goal positions. In order to avoid collision with other agents, each agent deviates from its nominal trajectory and successfully reaches its goal position.

where $\mathbf{1}\{\cdot\}$ is the indicator function, $d(x_k^i, x_k^j)$ is the Euclidean distance between the two agents, and d_m is the maximum threshold distance between the agents after which the agents start incurring the collision avoidance cost. We choose the value of d_m to be 2 m. Since the game is a w-potential game, we can employ the result from Theorem 1. We use the iLQR algorithm described in Section VI to minimize the resulting single-agent optimal control problem associated with minimizing the potential function. We choose coefficients of interagent costs (c^i's) such that Agent 1 incurs a high cost for being close to other agents. The resulting trajectories are plotted in Fig. 2. Since Agent 1 has a higher coefficient of interagent cost compared to the other agents, it yields more to the other agents. It deviates more compared to other agents from its nominal path when moving toward its goal location. This scenario can be considered a highway scenario where two agents are more aggressive while Agent 1 moves cautiously around such aggressive drivers. This shows that our algorithm is able to handle such real-life asymmetric situations.

Furthermore, to study constrained settings, we consider four agents sitting on the corners of a square of length 3 m such that their goal positions are the opposite diagonal ends of the square (see Fig. 7). Each agent wants to reach its goal position while avoiding collisions with others. We consider discrete-time unicycle dynamics similar to (45) to model each agent i. Suppose the desired position of each agent i is x_f^i. We consider the cost function of each agent i to be

$$J^i(x,u) = \frac{1}{2}\left(x_T^i - x_f^i\right)^\top Q_f^i \left(x_T^i - x_f^i\right) + \sum_{k=0}^{T-1} \frac{1}{2}\left(x_k^i - x_f^i\right)^\top Q^i \left(x_k^i - x_f^i\right) + \frac{1}{2} {u^i}_k^\top C^i u_k^i \tag{47}$$

where Q^i and C^i are diagonal matrices determining the deviation cost from the reference position and control costs, respectively. The matrix Q_f^i determines the cost of deviating from the desired position at the final time instance. To enforce collision avoidance among the agents, we consider a minimum separation constraint between the agents. We require the distance between any pair of agents to be greater than or equal to some threshold distance $d_{\text{collision}}$. Therefore, we impose collision avoidance constraints of the following form between any pair of agents

$$g_{ij}(x_k^i, x_k^j, k) := -d(x_k^i, x_k^j) + d_{\text{collision}} \leq 0 \tag{48}$$

where $d(x_k^i, y_k^i)$ is the distance between agents i and j at time step k. We consider the value of $d_{\text{collision}}$ to be 0.3 m. Furthermore, we consider additional constraints for bounding the actions of every each agent:

$$g_i(u_k^i, k) := |u_k^i| - u_{\text{bound}} \leq 0 \tag{49}$$

where u_{bound} is the bound on actions. We consider the bound u_{bound} to be 3 m/s for the linear velocity and 3 rd/s for the angular velocity. All these constraints can be concatenated to obtain the constraint vector $g(x_k, u_k, k)$. It is straightforward to see that the game is an instance of a constrained potential dynamic game. Therefore, we apply the results from Theorem 1 to obtain optimal trajectories. The simulation of the resulting trajectories is demonstrated in Fig. 7, where the agents manage to successfully avoid collisions with one another while reaching their designated goal locations. They exhibit intuitive trajectories, yielding and coordinating their motions to avoid collisions. Similar trajectories can be generated for various initial conditions. We further compare the solve time of our algorithm with the state-of-the-art constrained game solvers such as Algames [48] and PATH solver [49]. In order to utilize the PATH solver, we use the method provided in [50] to cast the game as a set of complementarity problems, which are then solved using the PATH solver. We generate 200 random sets of initial conditions for the four agents, and for each set of initial conditions, we run our algorithm, PATH solver, and Algames. Furthermore, we provide the same initializations for all the methods. We measure the solution time of both methods. The histograms of the solve time for the three methods are plotted in Fig. 8. The solve time of our method is on average 26.15 ± 34.30 ms while the average solve time of Algames was 577.35 ± 27.70 ms.

While working with the PATH solver, we observed that sometimes it is difficult for the PATH solver to converge for highly symmetrical problems such as in Fig 7. Because breaking symmetry becomes difficult without smart initializations when

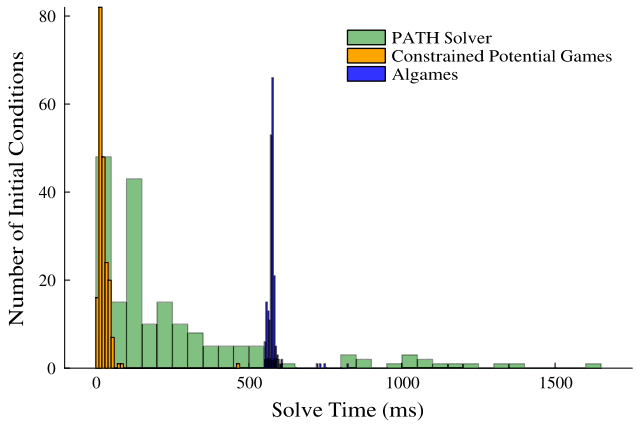


Fig. 8. Histograms for the comparison of the solve time of Constrained Potential Games with Algames. Constrained Potential Games has an average solve time of 26.15 ms in contrast to the PATH solver and Algames, which have an average solve time of 248.21 and 577.35 ms, respectively. Our approach is approximately 10 times faster than the PATH solver and 20 times faster than Algames.

solving complementarity problems. Therefore, we observe that out of 200 initial conditions, the PATH solver converges for 189. The solve time is generally very high for the runs where the PATH solver doesn't converge. Therefore, we only consider the converged times for our solve time computations. The solve time for the PATH-solver is 248.21 ± 302.05 ms. As Fig. 8 shows, our method is significantly faster than the PATH solver and Algames, and for most of the initial conditions, provides a solution in 20 ms while for Algames, the solve time is always greater than 500 ms. Furthermore, the variance in solve time for the PATH solver is really high. Our method is about 10 times faster than the PATH solver and 20 times faster than Algames.

VIII. Experiments

To demonstrate the performance of our proposed approach, we considered a hardware experiment where two Crazyflie quadrotors are navigating in a room shared with two humans. We require the quadrotors to transport a rigid rod with length 0.5m to enforce complicated task constraints. This implies that the quadrotors must maintain a fixed given distance from each other at all time steps, i.e., they must always satisfy an equality constraint. We further assume that agents are subject to collision avoidance constraints. This implies that quadrotors need to fulfill both equality and inequality constraints at each time step. The humans and quadrotors have designated initial and goal positions. The humans are asked to walk to their destinations, and the quadrotors need to carry the rod to the designated location while avoiding collisions with the humans. The quadrotors use our trajectory optimization algorithm to move from their initial positions to goal positions. The state vector of each quadrotor $i \in \{1, 2\}$, at each time step k, is

$$x_k^i = [p_{x,k}^i,\, p_{y,k}^i,\, p_{z,k}^i,\, \phi_k^i,\, \theta_k^i,\, \psi_k^i]$$

which consists of its position $([p_{x,k}^i,\, p_{y,k}^i,\, p_{z,k}^i])$ and orientation $([\phi_k^i,\, \theta_k^i,\, \psi_k^i])$. We model each quadrotor as a 6 DOF integrator, i.e., each quadrotor dynamics are given by $\dot{x}_k^i = u_k^i$, where $u_k^i = [u_{1,k}^i,\, u_{2,k}^i,\, u_{3,k}^i,\, u_{4,k}^i,\, u_{5,k}^i,\, u_{6,k}^i]$ is the control input. The first three entries of u_k^i correspond to the linear velocities, and the last three entries correspond to angular velocities. This design choice is based on the fact that we send waypoints as commands to Crazyflies through the Crazyswarm package [51], and they track those waypoints with an in-place low-level controller. Similar to [31], we model the humans via unicycle dynamics, except for the fact that there is a constant height r^i associated with each human $i, i \in \{1, 2\}$. We consider the human state vector to be

$$x_k^i = [p_{x,k}^i,\, p_{y,k}^i,\, r^i,\, \theta_k^i]$$

where $p_{x,k}^i,\, p_{y,k}^i$ denote the x, y coordinates of human i and θ_k^i denote the orientation of human i (yaw), all at time step k. We assume the z-coordinate of the human is located midway from the ground, i.e., $r^i \equiv 1$. We consider the cost functions of quadrotors to be of the same form as in (47), where the reference trajectory for each agent is a straight line connecting their initial and goal locations. Furthermore, in the case of humans, we assume that the goal positions of humans are known through which we can identify the desired reference trajectory of the human. Based on that, we assume that the human's cost function is also the same as in (47).[1] We assume that humans are rational, and while planning their motion in the environment, they also yield to each other and drones by satisfying constraints. We consider the collision avoidance constraint between the two humans to be the same as (48). For the collision avoidance constraint between each quadrotor and each human, we assume that each human occupies a cylinder of height 2 m with radius $\sqrt{0.4}$ m, centered at $(p_{x,k}^i,\, p_{y,k}^i,\, r^i)$. The maximum linear velocity for the quadrotors is 1.2 m/s, and we assume that humans have a maximum linear velocity of 1.5 m/s.

We implement our method in a receding horizon fashion to account for the uncertainties in human actions and the potential mismatch between the humans' cost functions and the drones' assumption of the humans' cost functions to some extent. However, due to the virtue of the assumptions, our method will not work well in case the cost functions are drastically different from what the drones assume. We set the planning horizon to be 0.5 s with a step size of 0.1 s. We solve the underlying optimization problem with do-mpc [7], which models the problem symbolically with CasADi [53] and solves them with IPOPT [54].

The resulting trajectories of our experiment are plotted in Fig 1. As shown in the figure, when the two quadrotors are relatively far from the two humans, they move straight toward their designated positions. When humans are nearby, the quadrotors can simultaneously change their orientations and the orientation of the rod to avoid collisions with humans while carrying the rod and maintaining the equality constraints.

IX. Conclusion

In conclusion, this study addresses the challenges posed by planning and decision-making in multiagent interactions. The

[1] We acknowledge that the drones in our experiment may not have the perfect knowledge of the cost functions of humans. Therefore, typically we obtain the cost functions of other agents through inverse methods such as inverse reinforcement learning [52] or inverse game theoretic methods [36].

computational complexity of reasoning about the actions and intentions of other agents, along with the diverse objectives of the agents involved, makes finding solutions a difficult task. By leveraging concepts from dynamic noncooperative game theory and constrained potential games, we proposed a novel approach to simplify the computation of OLGNE. The key idea is to identify structures in realistic multiagent planning scenarios that can be transformed into WCPDGs. Under specific cost structures, the problem can be formulated as a WCPDG, and the OLGNE can be obtained by solving a single constrained optimal control problem. This transformation significantly reduces the computational burden compared to solving a set of coupled constrained optimal control problems. We showcase the capabilities of our method through extensive numerical simulations and hardware experiments. We show that our method is significantly faster than state-of-the-art game solvers.

In the future, we aim to expand the algorithm's capabilities to encompass more general cost functions, even accommodating different functional forms for inter-agent costs. We plan to extend the scope of our cost functions to include highly versatile structures, such as neural networks. By doing so, we intend to merge inverse cost learning methods with potential games, enabling the application of these ideas to real-world robotic systems.

APPENDIX

A. Proof of Theorem 1

Proof: This theorem was proven in [43] for exact potential games with an infinite horizon when $T = \infty$. Here, we state the proof for WCPDG with finite horizons. It should be noted that as per [43], Assumption 2 is required to ensure that the KKT conditions hold at the optimal point and that there exist feasible dual variables (see [55], Prop. 3.3.8). With that, the Lagrangian for each agent $i \in \mathcal{N}$ can be written as

$$\mathcal{L}(x_k, u_k, \lambda^i, \mu^i) = L^i(x_T) + \mu_T^{i^\top} g(x_T) + \sum_{k=0}^{T-1} \Big(L_k^i(x_k, u_k) + \lambda_k^{i^\top} \left(f(x_k, u_k) - x_{k+1} \right) + \mu_k^{i^\top} g(x_k, u_k) \Big) \tag{50}$$

where $\lambda^i = \{\lambda_k^i\}_{k\in\{0,1,\ldots,T\}}$ and $\mu^i = \{\mu_k^i\}_{k\in\{0,1,\ldots,T\}}$ are the vector of Lagrange multiplies for agent i. For every agent $i \in \mathcal{N}$, the KKT conditions for the game $\mathcal{G}$ can be written as

$$\frac{\partial L_k^i(x_k, u_k)}{\partial x_k^i} + \lambda_k^{i^\top} \frac{\partial f(x_k, u_k)}{\partial x_k^i} + \mu_k^{i^\top} \frac{\partial g(x_k, u_k)}{\partial x_k^i} - \lambda_{k-1}^i = 0 \tag{51}$$

$$\frac{\partial L_k^i(x_k, u_k)}{\partial u_k^i} + \lambda_k^{i^\top} \frac{\partial f(x_k, u_k)}{\partial u_k^i} + \mu_k^{i^\top} \frac{\partial g(x_k, u_k, k)}{\partial u_k^i} = 0 \tag{52}$$

$$x_{k+1} = f(x_k, u_k), g(x_k, x_k) \leq 0 \; \forall k \in \{0, 1, \ldots, T-1\} \tag{53}$$

$$\mu_k^i \leq 0, \quad \mu_k^{i^\top} g(x_k, u_k) = 0, \; \forall \, 0 \leq k \leq T-1 \tag{54}$$

$$\frac{\partial L_T^i(x_T)}{\partial x_T^i} + \mu_T^{i^\top} \frac{\partial g(x_T)}{\partial x_T^i} = 0 \tag{55}$$

$$g(x_T) \leq 0, \mu_T^i \leq 0, \quad \mu_T^{i^\top} g(x_T) = 0. \tag{56}$$

To compute the KKT conditions for the multivariate constrained optimal control, we write the Lagrangian of (17) as

$$\mathcal{L}^P(x_k, u_k, \xi_k) = L_T(x_T) + \delta_T^\top g(x_T) + \sum_{k=0}^{T-1} \Big(L_k(x_k, u_k) + \xi_k{}^\top \left(f(x_k, u_k) - x_{k+1} \right) + \delta_k^\top g(x_k, u_k) \Big) \tag{57}$$

where ξ_k is the vector of Lagrange multipliers at time-step k. Therefore, its KKT conditions can be written as

$$\frac{\partial L_k(x_k, u_k)}{\partial x_k^i} + \xi_k^\top \frac{\partial f(x_k, u_k)}{\partial x_k^i} + \delta_k^\top \frac{\partial g(x_k, u_k)}{\partial x_k^i} - \xi_{k-1} = 0 \tag{58}$$

$$\frac{\partial L_k(x_k, u_k)}{\partial u_k^i} + \xi_k^\top \frac{\partial f(x_k, u_k)}{\partial u_k^i} + \delta_k^\top \frac{\partial g(x_k, u_k, k)}{\partial u_k^i} = 0 \tag{59}$$

$$x_{k+1} = f(x_k, u_k, k), g(x_k, x_k) \leq 0 \; \forall k \in \{0, 1, \ldots, T-1\} \tag{60}$$

$$\delta_k \leq 0, \quad \delta_k^\top g(x_k, u_k) = 0, \; \forall \, 0 \leq k \leq T-1 \tag{61}$$

$$\frac{\partial L_T(x_T)}{\partial x_T^i} + \delta_T^\top \frac{\partial g(x_T)}{\partial x_T^i} = 0 \tag{62}$$

$$g(x_T) \leq 0, \delta_T \leq 0, \quad \delta_T^\top g(x_T) = 0. \tag{63}$$

For each time step, we can multiply the KKT optimality conditions by a fixed positive number (w_k^i) on both sides and still the optimality conditions will remain equivalent to the original conditions. Therefore, the equivalent version of optimality conditions for (17) will be as follows:

$$w_k^i \Bigg(\frac{\partial L_k(x_k, u_k)}{\partial x_k^i} + \xi_k^\top \frac{\partial f(x_k, u_k)}{\partial x_k^i} \tag{64}$$

$$+ \delta_k^\top \frac{\partial g(x_k, u_k)}{\partial x_k^i} - \xi_{k-1} \Bigg) = 0 \tag{65}$$

$$w_k^i \left(\frac{\partial L_k(x_k, u_k)}{\partial u_k^i} + \xi_k^\top \frac{\partial f(x_k, u_k)}{\partial u_k^i} + \delta_k^\top \frac{\partial g(x_k, u_k, k)}{\partial u_k^i} \right) = 0 \tag{66}$$

$$x_{k+1} = f(x_k, u_k, k), g(x_k, x_k) \leq 0 \; \forall k \in \{0, 1, \ldots, T-1\} \tag{67}$$

$$\delta_k \leq 0, \quad \delta_k^\top g(x_k, u_k) = 0, \; \forall \, 0 \leq k \leq T-1 \tag{68}$$

$$w_T^i \left(\frac{\partial L_T(x_T)}{\partial x_T^i} + \delta_T^\top \frac{\partial g(x_T)}{\partial x_T^i} \right) = 0 \tag{69}$$

$$g(x_T) \leq 0, \delta_T \leq 0, \quad \delta_T^\top g(x_T) = 0. \tag{70}$$

Thus, in order for multivariate optimal control problem (17) and game $\mathcal{G}$ to have same optimality conditions the following must

hold:

$$\frac{\partial L_k^i(x_k^i, u_k)}{\partial x_k^i} = w_k^i \left(\frac{\partial L_k(x_k, u_k)}{\partial x_k^i} \right) \tag{71}$$

$$\frac{\partial L_k^i(x_k^i, u_k)}{\partial u_k^i} = w_k^i \left(\frac{\partial L_k(x_k, u_k)}{\partial u_k^i} \right) \tag{72}$$

$$\forall i \in \mathcal{N},\ k = 0, 1, \ldots, T-1;\ \text{and}$$

$$\frac{\partial L_T^i(x_T^i)}{\partial x_T^i} = w_T^i \left(\frac{\partial L_T(x_T, T)}{\partial x_T^i} \right) \tag{73}$$

$$\lambda_k^i = w_k^i \xi_k,\ \mu_k^i = \delta_k\ \forall i \in \mathcal{N}. \tag{74}$$

When conditions (71)–(73) are satisfied, Lemma-1 holds true. Since, Lemma-1 is necessary and sufficient condition for a game $\mathcal{G} := (\mathcal{N}, \{\Gamma_i\}_{i\in\mathcal{N}}, \{J_i\}_{i\in\mathcal{N}}, \{\mathcal{C}_k\}_{k\in\{0,\ldots,T\}}, f, x_0)$ to be a WCPDG, a solution to the (17) will be an open-loop Nash equilibria of $\mathcal{G}$. Readers are referred to proof of [43, Theorem 1] for further details. □

B. Exempt Determination Involving Human Subjects

This experiment was conducted while all the authors were working at the University of Illinois Urbana-Champaign (UIUC). The University of Illinois at Urbana-Champaign Office for the Protection of Research Subjects (OPRS) authorized the authors to use human subjects and determined that the criteria for exemption had been met. The protocol number for the same is 22107.

REFERENCES

[1] T. Başar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, Philadelphia, PA, USA: SIAM, 1998.

[2] R. Isaacs, *Differential Games: A Mathematical Theory With Applications to Warfare and Pursuit, Control and Optimization*. New York, NY, USA: Courier Corporation, 1999.

[3] D. Monderer and L. S. Shapley, "Potential games," *Games Econ. Behav.*, vol. 14, no. 1, pp. 124–143, 1996.

[4] T. Kavuncu, A. Yaraneri, and N. Mehr, "Potential ILQR: A potential-minimizing controller for planning multi-agent interactive trajectories," 2021, *arXiv:2107.04926*.

[5] M. Bhatt, A. Yaraneri, and N. Mehr, "Efficient constrained multi-agent interactive planning using constrained dynamic potential games," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Detroit, MI, USA, 2023, pp. 7303–7310, doi: 10.1109/IROS55552.2023.10342328.

[6] T. A. Howell, B. E. Jackson, and Z. Manchester, "Altro: A fast solver for constrained trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 7674–7679.

[7] S. Lucia, A. Tătulea-Codrean, C. Schoppmeyer, and S. Engell, "Rapid development of modular and sustainable nonlinear model predictive control solutions," *Control Eng. Pract.*, vol. 60, pp. 51–62, 2017.

[8] W. Li and E. Todorov, "Iterative linear quadratic regulator design for nonlinear biological movement systems," in *Proc. 1st Int. Conf. Informat. Control, Autom. Robot.*, SciTePress, 2004, vol. 2, pp. 222–229.

[9] J. Yu and S. M. LaValle, "Optimal multirobot path planning on graphs: Complete algorithms and effective heuristics," *IEEE Trans. Robot.*, vol. 32, no. 5, pp. 1163–1177, Oct. 2016.

[10] S. Tang, J. Thomas, and V. Kumar, "Hold or take optimal plan (HOOP): A quadratic programming approach to multi-robot trajectory generation," *Int. J. Robot. Res.*, vol. 37, no. 9, pp. 1062–1084, 2018.

[11] F. Augugliaro, A. P. Schoellig, and R. D'Andrea, "Generation of collision-free trajectories for a quadrocopter fleet: A sequential convex programming approach," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2012, pp. 1917–1922.

[12] V. R. Desaraju and J. P. How, "Decentralized path planning for multi-agent teams with complex constraints," *Auton. Robots*, vol. 32, pp. 385–403, 2012.

[13] C. Toumieh and A. Lambert, "Decentralized multi-agent planning using model predictive control and time-aware safe corridors," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 11110–11117, Oct. 2022.

[14] D. Zhou, Z. Wang, S. Bandyopadhyay, and M. Schwager, "Fast, on-line collision avoidance for dynamic vehicles using buffered Voronoi cells," *IEEE Robot. Automat. Lett.*, vol. 2, no. 2, pp. 1047–1054, Apr. 2017.

[15] L. Lv, S. Zhang, D. Ding, and Y. Wang, "Path planning via an improved DQN-based learning policy," *IEEE Access*, vol. 7, pp. 67319–67330, 2019.

[16] H. Qie, D. Shi, T. Shen, X. Xu, Y. Li, and L. Wang, "Joint optimization of multi-UAV target assignment and path planning based on multi-agent reinforcement learning," *IEEE access*, vol. 7, pp. 146264–146272, 2019.

[17] Z. Liu, B. Chen, H. Zhou, G. Koushik, M. Hebert, and D. Zhao, "Mapper: Multi-agent path planning with evolutionary reinforcement learning in mixed dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 11748–11754.

[18] S. H. Semnani, H. Liu, M. Everett, A. De Ruiter, and J. P. How, "Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3221–3226, Apr. 2020.

[19] A. P. Vinod, S. Safaoui, A. Chakrabarty, R. Quirynen, N. Yoshikawa, and S. Di Cairano, "Safe multi-agent motion planning via filtered reinforcement learning," in *Proc. 2022 Int. Conf. Robot. Automat.*, 2022, pp. 7270–7276.

[20] G. Papoudakis, F. Christianos, A. Rahman, and S. V. Albrecht, "Dealing with non-stationarity in multi-agent deep reinforcement learning," 2019, *arXiv:1906.04737*.

[21] C. S. De Witt et al., "Is independent learning all you need in the starcraft multi-agent challenge?," 2020, *arXiv:2011.09533*.

[22] H. Kretzschmar, M. Spies, C. Sprunk, and W. Burgard, "Socially compliant mobile robot navigation via inverse reinforcement learning," *Int. J. Robot. Res.*, vol. 35, no. 11, pp. 1289–1307, 2016.

[23] S. Parsons and M. Wooldridge, "Game theory and decision theory in multi-agent systems," *Auton. Agents Multi-Agent Syst.*, vol. 5, pp. 243–254, 2002.

[24] D. Sadigh, S. Sastry, S. A. Seshia, and A. D. Dragan, "Planning for autonomous cars that leverage effects on human actions," in *Proc. Robot.: Sci. Syst.*, Ann Arbor, MI, USA, 2016, vol. 2, pp. 1–9.

[25] J. F. Fisac, E. Bronstein, E. Stefansson, D. Sadigh, S. S. Sastry, and A. D. Dragan, "Hierarchical game-theoretic planning for autonomous vehicles," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 9590–9596.

[26] A. Turnwald and D. Wollherr, "Human-like motion planning based on game theoretic decision making," *Int. J. Social Robot.*, vol. 11, pp. 151–170, 2019.

[27] R. Spica, E. Cristofalo, Z. Wang, E. Montijano, and M. Schwager, "A real-time game theoretic planner for autonomous two-player drone racing," *IEEE Trans. Robot.*, vol. 36, no. 5, pp. 1389–1403, Oct. 2020.

[28] M. Wang, Z. Wang, J. Talbot, J. C. Gerdes, and M. Schwager, "Game theoretic planning for self-driving cars in competitive scenarios," in *IEEE Trans. Robot.*, vol. 37, no. 4, pp. 1313–1325, Aug. 2021, doi: 10.1109/TRO.2020.3047521.

[29] Z. Wang, R. Spica, and M. Schwager, "Game theoretic motion planning for multi-robot racing," in *Proc. Distrib. Auton. Robot. Syst.: 14th Int. Symp.*, Springer, 2019, pp. 225–238.

[30] K. Miller and S. Mitra, "Multi-agent motion planning using differential games with lexicographic preferences," in *Proc. IEEE 61st Conf. Decis. Control*, 2022, pp. 5751–5756.

[31] D. Fridovich-Keil, E. Ratner, L. Peters, A. D. Dragan, and C. J. Tomlin, "Efficient iterative linear-quadratic approximations for nonlinear multi-player general-sum differential games," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 1475–1481.

[32] W. Schwarting, A. Pierson, S. Karaman, and D. Rus, "Stochastic dynamic games in belief space," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 2157–2172, Dec. 2021.

[33] S. Le Cleac'h, M. Schwager, and Z. Manchester, "Algames: A fast augmented lagrangian solver for constrained dynamic games," *Auton. Robots*, vol. 46, no. 1, pp. 201–215, 2022.

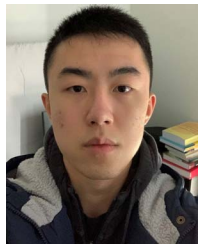
[34] F. Laine, D. Fridovich-Keil, C.-Y. Chiu, and C. Tomlin, "The computation of approximate generalized feedback nash equilibria," *SIAM J. Optim.*, vol. 33, no. 1, pp. 294–318, 2023.

[35] M. Wang, N. Mehr, A. Gaidon, and M. Schwager, "Game-theoretic planning for risk-aware interactive agents," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 6998–7005.

[36] N. Mehr, M. Wang, M. Bhatt, and M. Schwager, "Maximum-entropy multi-agent dynamic games: Forward and inverse solutions," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1801–1815, Jun. 2023.
[37] M. Chahine, R. Firoozi, W. Xiao, M. Schwager, and D. Rus, "Intention communication and hypothesis likelihood in game-theoretic motion planning," *IEEE Robot. Automat. Lett.*, vol. 8, no. 3, pp. 1223–1230, Mar. 2023.
[38] R. W. Rosenthal, "A class of games possessing pure-strategy Nash equilibria," *Int. J. Game Theory*, vol. 2, no. 1, pp. 65–67, 1973.
[39] A. Fonseca-Morales and O. Hernández-Lerma, "Potential differential games," *Dyn. Games Appl.*, vol. 8, pp. 254–279, 2018.
[40] G. Arslan, J. R. Marden, and J. S. Shamma, "Autonomous vehicle-target assignment: A game-theoretical formulation," *J. Dyn. Syst., Meas., Control*, vol. 129, no. 5, pp. 584–596, 2007. [Online]. Available: https://doi.org/10.1115/1.2766722
[41] J. R. Marden, G. Arslan, and J. S. Shamma, "Cooperative control and potential games," *IEEE Trans. Syst., Man, Cybern., Part B*, vol. 39, no. 6, pp. 1393–1407, Dec. 2009.
[42] S. Zazo, J. Zazo, and M. Sánchez-Fernández, "A control theoretic approach to solve a constrained uplink power dynamic game," in *Proc. IEEE 22nd Eur. Signal Process. Conf.*, 2014, pp. 401–405.
[43] S. Zazo, S. V. Macua, M. Sánchez-Fernández, and J. Zazo, "Dynamic potential games with constraints: Fundamentals and applications in communications," *IEEE Trans. Signal Process.*, vol. 64, no. 14, pp. 3806–3821, Jul. 2016.
[44] Z. Williams, J. Chen, and N. Mehr, "Distributed potential ILQR: Scalable game-theoretic trajectory planning for multi-agent interactions," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023.
[45] Y. Jia, M. Bhatt, and N. Mehr, "Rapid: Autonomous multi-agent racing using constrained potential dynamic games," in *Proc. Eur. Control Conf.*, 2023, pp. 1–8.
[46] J. Li, C.-Y. Chiu, L. Peters, S. Sojoudi, C. Tomlin, and D. Fridovich-Keil, "Cost inference for feedback dynamic games from noisy partial state observations and incomplete trajectories," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.*, Richland, SC, 2023, pp. 1062–1070.
[47] L. Peters, D. Fridovich-Keil, C. J. Tomlin, and Z. N. Sunberg, "Inference-based strategy alignment for general-sum differential games," in *Proc. 19th Int. Conf. Auton. Agents Multiagent Syst.*, Richland, SC, 2020, pp. 1037–1045.
[48] S. L. Cleac'h, M. Schwager, and Z. Manchester, "Algames: A fast solver for constrained dynamic games," 2019, *arXiv:1910.09713*.
[49] S. P. Dirkse and M. C. Ferris, "The path solver: A nommonotone stabilization scheme for mixed complementarity problems," *Optim. Methods Softw.*, vol. 5, no. 2, pp. 123–156, 1995.
[50] X. Liu, L. Peters, and J. Alonso-Mora, "Learning to play trajectory games against opponents with unknown objectives," *IEEE Robot. Automat. Lett.*, vol. 8, no. 7, pp. 4139–4146, Jul. 2023.
[51] J. A. Preiss∗, W. Hönig∗, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 3299–3304, software available at https://github.com/USC-ACTLab/crazyswarm. [Online]. Available: https://doi.org/10.1109/ICRA.2017.7989376
[52] S. Russell, "Learning agents for uncertain environments," in *Proc. 11th Annu. Conf. Comput. Learn. Theory*, 1998, pp. 101–103.
[53] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi–A software framework for nonlinear optimization and optimal control," *Math. Program. Comput.*, vol. 11, no. 1, pp. 1–36, 2019.
[54] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006.
[55] D. P. Bertsekas, "Nonlinear programming," *J. Oper. Res. Soc.*, vol. 48, no. 3, pp. 334–334, 1997.

Maulik Bhatt (Graduate Student Member, IEEE) received the Interdisciplinary Dual Degree (Bachelor's degree in aerospace engineering + Master's degree in systems and control engineering) from the Indian Institute of Technology Bombay, Mumbai, India, in 2021.

He is currently a Ph.D. student with the Department of Mechanical Engineering at UC Berkeley, Berkeley, CA, USA. Before joining Berkeley, he was a Ph.D. student with the Department of Aerospace Engineering, University of Illinois Urbana-Champaign. He is interested in leveraging game theory, stochastic control, and machine learning to enable efficient robotic multiagent interactions and safe motion planning. During his undergraduate studies, he worked in the area of state estimation using geometric control and variational principles.



Yixuan Jia received the B.S. degrees in computer engineering and mathematics from the University of Illinois Urbana-Champaign, Champaign, IL, USA, in 2023. He is currently working toward the M.S. degree in aeronautics and astronautics with the Massachusetts Institute of Technology, Cambridge, MA, USA.

During his undergraduate studies, he studied multiagent planning, safety verification of autonomous systems, and control theory. His current research interests include perception, localization, and planning for autonomous navigation in unstructured environments.

Negar Mehr (Member, IEEE) received the B.S. degree from the Sharif University of Technology, Tehran, Iran, and the Ph.D. degree from UC Berkeley, Berkeley, CA, USA, in 2013 and 2019, respectively, both in mechanical engineering.

From 2019 to 2020, she was a Postdoctoral Scholar with Stanford Aeronautics and Astronautics Department. She is an Assistant Professor with the Mechanical Engineering Department, University of California, Berkeley, CA, USA. Before joining Berkeley, she was an Assistant Professor in aerospace engineering with the University of Illinois Urbana-Champaign. Her research interest includes learning and control algorithms for interactive robots, robots that can safely and intelligently interact with other agents.

Dr. Mehr is the recipient of the NSF CAREER award in 2022. Her Ph.D. dissertation was awarded the 2020 IEEE ITSS Best Ph.D. Dissertation Award.